

DA2304: Introduction to Computer Systems

Assignment 3: Jack Compiler & 2D Convolution

Data Science and AI Department (DSAI)
Indian Institute of Technology Madras

Instructor: Dr. Patanjali SLPSK (patanjali@dsai.iitm.ac.in)

Release Date: May 6, 2026

Deadline: May 20, 2026, 11:59 PM

Introduction

In Assignment 2, you climbed the abstraction ladder from raw Hack assembly to the stack-based Virtual Machine, building a VM Translator that converts `.vm` commands into native Hack assembly. You also analyzed how function-calling strategies impact performance for matrix operations.

In **Part 1** of this assignment, you will take the next step and write a high-level program in the **Jack** programming language. Specifically, you will implement a 2D **Convolution** (Conv) filter — one of the most fundamental primitives in modern Computer Vision and Deep Learning — as a Jack class.

In **Part 2**, you will build the compiler frontend: a **Tokenizer** that produces an XML stream of lexical tokens, and a **Parser** (syntax analyzer) that validates the token stream against the Jack grammar and emits the corresponding VM code. Together with your Assignment 2 VM Translator, this completes the full compilation pipeline:

Jack source $\longrightarrow$ Tokens (XML) $\longrightarrow$ VM code $\longrightarrow$ Hack assembly $\longrightarrow$ Hack binary

Prerequisites

1. Working VM Translator from Assignment 2.
2. Familiarity with the Jack programming language (Chapter 9 of *The Elements of Computing Systems*).
3. Understanding of compilation theory: lexical analysis, recursive-descent parsing, and the Jack grammar (Fig. 10.5 of the textbook).
4. Basic understanding of 2D Convolution operations and stride/padding semantics.
5. Proficiency in Python for implementing the Tokenizer and Parser.

Part 1: The Jack Convolution Program

1 Exercise 1: 2D Convolution — Background

A 2D Convolution slides a small filter (also called a *kernel*) across a larger input matrix. At each position, it computes the element-wise product between the filter and the overlapping window of the input, and sums the result into a single scalar. That scalar becomes one entry of the output matrix.

For an input of size $N \times N$, a filter of size $F \times F$, padding P , and stride S , the output dimension is:

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

For this assignment, use $F = 3$ and $P = 0$. The stride S is configurable (1 or 2).

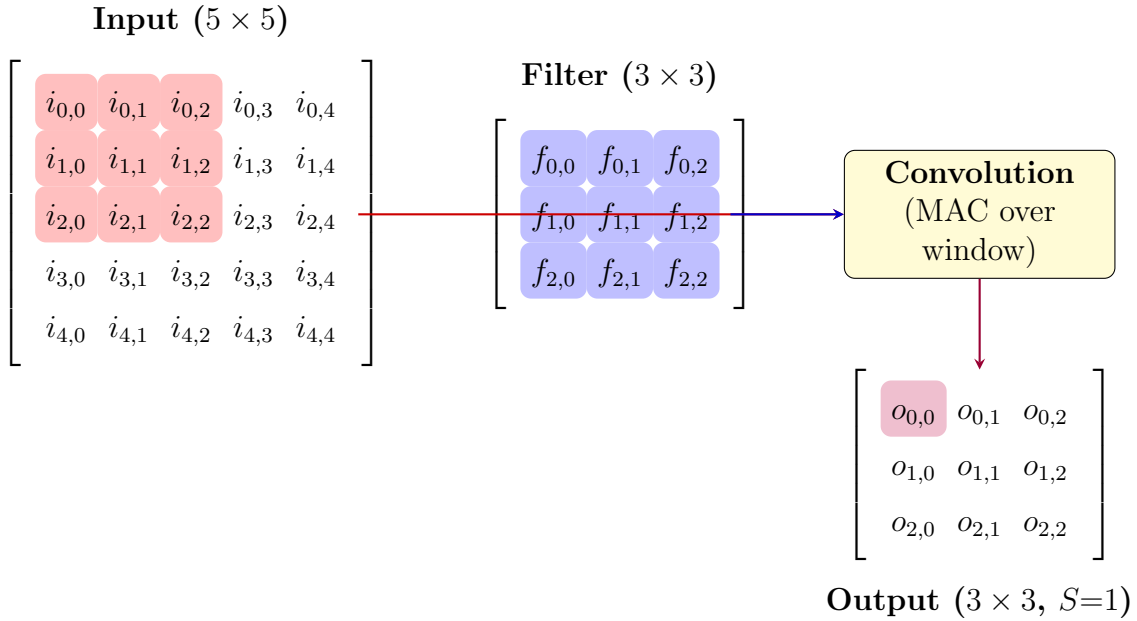


Figure 1: Sliding a 3×3 filter over a 5×5 input. Each window position produces one scalar in the output matrix.

2 Exercise 2: The Conv Class in Jack

Implement a class `Conv` in the Jack language that performs 2D convolution. Save it as `Conv.jack`.

2.1 Class Skeleton

The class must follow the structure below. You are allowed to add additional fields, static variables, methods, and helper functions on top of those listed — **provided you justify each addition** in your report.

Listing 1: Skeleton of the `Conv` class

```

1 class Conv {
2     static Array ArrayFilter;           // 3x3 filter, flattened to
        ↳ length 9
3     static int stride;                  // initialised in constructor (1
        ↳ or 2)
4
5     field Array ArrayInput;             // input matrix, flattened
6     field Array ArrayOutput;           // output matrix, flattened
7     field int ArraySize;                // side length N of input (NxN)
8     // Filter is always of size 3.
9
10    /** Constructor: initialise stride and allocate output. */
11    constructor Conv new(int N) { ... }
12
13    /** Initialise the 3x3 filter with given values. */
14    method void initFilter(Array values) { ... }
15
16    /** Read input matrix entries from the caller. */
17    method void getArrayInput(Array input, int size) { ... }
18
19    /** Compute output dimension using the conv formula. */
20    method int calcOutputDim() { ... }
21
22    /** Run the full convolution and store result in ArrayOutput. */
23    method void conv2d() { ... }
24
25    /** Print the output matrix to screen. */
26    method void printOutput() { ... }
27
28    /** Free allocated memory. */
29    method void dispose() { ... }
30 }

```

2.2 Functional Requirements

1. **Constructor:** Allocate `ArrayInput`, `ArrayOutput`, and `ArrayFilter`. Set `stride` to either 1 or 2 (your choice; document it in the report).
2. **initFilter:** Populate the static `ArrayFilter` with 9 integer values supplied by the caller. Use a meaningful kernel (e.g., edge detector, blur, or identity) and document your choice.
3. **getArrayInput:** Accept an input matrix (flattened array) and its side length, and store it as the object's input.
4. **calcOutputDim:** Implement the formula

$$O = \left\lfloor \frac{N - 3 + 2 \cdot P}{S} \right\rfloor + 1$$

using Jack integer arithmetic. Note: Jack division (`/`) is integer division, which gives you the floor for free on non-negative operands.

5. **convolve:** Slide the filter across the input with the configured stride. For each window, compute the MAC (multiply-accumulate) and store the resulting scalar into `ArrayOutput`. Bounds checking is required — if the input is too small for a single window, signal the error condition (e.g., set a flag in RAM, in the spirit of the `throw_error` convention from Assignment 2).
6. **printOutput:** Print the output matrix row-by-row using `Output.printInt` and `Output.printChar/Output.println`.

2.3 Driver Program (Main.jack)

Write a `Main` class whose `main` function:

- (a) Allocates a 5×5 input matrix with hand-picked values.
- (b) Creates a `Conv` object, initialises a 3×3 filter, runs `convolve()`, and calls `printOutput()`.
- (c) Disposes the object cleanly.

You are encouraged to additionally test on a 9×9 input for extra credit (5 points).

Part 2: The Jack Compiler Frontend

1 Exercise 1: The Tokenizer (Lexical Analyzer)

Write a Python module `JackTokenizer.py` that converts a `.jack` source file into a stream of tokens. Each token must be classified into exactly one of the following categories defined by the Jack language specification:

- **keyword** — e.g., `class`, `constructor`, `method`, `let`, `do`, `return`, `if`, `while`, `int`, `boolean`, `true`, `false`, `null`, `this`, etc.
- **symbol** — one of `{ }` `( )` `[ ]` `.` `,` `;` `+` `-` `*` `/` `&` `!` `=` `—`
- **integerConstant** — a decimal integer in the range `[0, 32767]`.
- **stringConstant** — a double-quoted sequence of characters (no newlines, no double-quotes).
- **identifier** — a sequence of letters, digits, and underscores not starting with a digit.

1.1 Required Output Format

The tokenizer must emit an XML file named `<ClassName>T.xml` (note the trailing T) of the form:

Listing 2: Example tokenizer output

```
1 <tokens>
2   <keyword> class </keyword>
3   <identifier> Conv </identifier>
4   <symbol> { </symbol>
5   <keyword> static </keyword>
6   <keyword> int </keyword>
7   <identifier> stride </identifier>
8   <symbol> ; </symbol>
9   ...
10 </tokens>
```

Important: Comments (`//`, `/* */`, `/** */`) and whitespace must be stripped. The five XML-unsafe symbols (`<`, `>`, `&`, `"`) must be escaped (`<`, `>`, `&`, `"`).

2 Exercise 2: The Parser (Syntax Analyzer + VM Code Generator)

Write a Python module `CompilationEngine.py` that performs recursive-descent parsing on the token stream and emits VM code. The grammar to be implemented is given in **Figure 10.5** of *The Elements of Computing Systems* (Nisan & Schocken, 2nd Edition). Your parser must implement *every* non-terminal in that grammar.

2.1 Two-Stage Deliverable

1. **Stage 1 (Parse Tree XML):** First, build a parser that emits an XML parse tree (`<ClassName>.xml`, no trailing T). This validates that the token stream is grammatically correct. Use the same XML-tag conventions as in Chapter 10 of the textbook.
2. **Stage 2 (VM Code Generation):** Extend the parser to a full **compilation engine** (Chapter 11) that emits valid `.vm` code. Maintain a symbol table with two scopes (class-level and subroutine-level) to map identifiers to memory segments (`static`, `this`, `argument`, `local`).

2.2 Compilation Targets

Your compiler must correctly translate at least the following constructs:

- Class and subroutine declarations (`constructor`, `function`, `method`).
- Variable declarations (`static`, `field`, `var`, parameter list).
- Control flow: `if`, `else`, `while`.
- Statements: `let`, `do`, `return`.
- Expressions with correct operator handling (Jack has *no* operator precedence — evaluation is left-to-right).
- Subroutine calls: method calls on `this`, calls on other objects, and function calls on classes.
- Array access using `THAT` via the `pointer` 1 segment.

2.3 End-to-End Pipeline

Wire your three components together in `JackCompiler.py`:

```
1 # Pipeline: Conv.jack --> ConvT.xml + Conv.xml + Conv.vm
2 tokenizer = JackTokenizer("Conv.jack")
3 tokens    = tokenizer.tokenize()           # also writes ConvT.xml
4 engine    = CompilationEngine(tokens)
5 engine.compile_class()                     # writes Conv.xml + Conv.vm
```

The emitted `Conv.vm` (and `Main.vm`) must run correctly when fed into your Assignment 2 VM Translator and executed on the Hack CPU emulator.

3 Exercise 3: Analysis Questions

Answer the following in your report:

1. **Tokenizer design:** Why is it important that the tokenizer handles `/* ... */` comments using a state machine rather than a simple regex on a single line? Give a concrete failure case.
2. **Symbol table:** Walk through the symbol-table state when the parser enters `Conv.convolve()`. Show the `kind`, `type`, and `index` for every identifier in scope at the point where the inner-loop accumulator is assigned.
3. **Compilation correctness:** For your 5×5 test case, manually trace the VM code emitted for the line `let ArrayOutput[k] = sum;` and explain how the `pointer 1 / that 0` idiom resolves to the correct RAM address.
4. **Convolution stride:** For a 9×9 input with $F = 3$ and $S = 2$, what is the output dimension? Do the values of $S = 1$ and $S = 2$ change the asymptotic VM-instruction count of `convolve()`? Explain.

Submission Instructions

1. **Report:** Theoretical answers, design justifications, and analysis questions go in `Name_RollNo.pdf`.
2. **Jack source:** Place `Conv.jack` and `Main.jack` in a folder named `jack/`.
3. **Compiler source:** Place your Python compiler (`JackTokenizer.py`, `CompilationEngine.py`, `SymbolTable.py`, `VMWriter.py`, `JackCompiler.py`) in a folder named `src/`, together with a `README.md` describing how to run the pipeline.
4. **Generated artefacts:** Include the generated `ConvT.xml`, `MainT.xml`, `Conv.xml`, `Main.xml`, `Conv.vm`, `Main.vm` in a folder named `out/`.
5. **Demonstration:** Include a screenshot or short text log showing the printed output of your 5×5 convolution running on the Hack CPU emulator. (A 9×9 run is encouraged but optional.)
6. **Integration:** Show that the VM files produced by this assignment, when fed into the Assignment-2 VM Translator, produce assembly that runs end-to-end on the Hack architecture.
7. Compress everything into a **ZIP archive** named:
`DA2304_Assignment3_RollNumber.zip`
8. Upload the archive to the course portal by May 20, 2026, 11:59 PM.
9. **Report Extra Credit (2 points):** In a section titled "Extra Credit", describe the most interesting bug you encountered while building the parser, and how you debugged it.

Good luck! For queries, post on the course forum or contact the TAs.